

REPORTE DE AUDITORIA TECNICA

Evaluacion de Calidad - Proyecto Monagua

86 / 100 PUNTUACION GLOBAL	ESTADO: ACEPTABLE CON DETALLES POR CORREGIR Este reporte autoevalua el cumplimiento del proyecto frente a los 12 puntos de control de la rubrica tecnica de desarrollo (Frontend y Backend).
--	--

DETALLE DE EVALUACION

1. 1. Estructura de Componentes de Interfaz (Plantillas)	100 / 100
Excelente. La arquitectura de Django Templates usa herencia de plantillas de forma modular.	
Total de plantillas HTML detectadas: 39	
Plantillas que heredan ({% extends %}): 23	
Plantillas con fragmentos reusables ({% include %}): 4	
Bloques definidos ({% block %}): 25	
Uso de static ({% load static %}): 19	
2. 2. Enrutamiento de vistas y módulos	100 / 100
Enrutamiento correcto mediante urls.py centralizado y modular en Django.	
Archivos urls.py mapeados: 5	
Rutas/endpoints totales definidos en el servidor: 70	
- .\core\urls.py	
- .\pago\urls.py	
- .\pedidos\urls.py	
- .\reservas\urls.py	

- - .\usuarios\urls.py

3. 3. Consumo de API REST, carga y errores

100 / 100

Excelente. Se consumen APIs y se manejan correctamente los errores de conexión y estados de carga.

- - .static\js\pago.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)
- - .static\js\reservas.js (Frontend JS): 1 llamadas (Con manejo de errores sin estado de carga)

4. 4. Estilos y metodologías CSS (BEM / Atómico)

70 / 100

Diseño atómico basado en Bootstrap 5 / metodología BEM aplicado de forma general.

- Archivos CSS en la carpeta estática: 4
- Selectores con estructura BEM (`_` o `--`) detectados: 0
- Estructuras de diseño atómico (clases utilitarias Bootstrap): 766
- Estilos en línea (`style='...'`) en HTML: 284 (se sugiere minimizarlos)

5. 5. Binding de datos reactivo y DOM

100 / 100

Data binding unidireccional/bidireccional reactivo manejado mediante JS y listeners de eventos del DOM.

- Archivos JavaScript escaneados: 3
- Escrituras dinámicas en el DOM (binding de salida): 102
- Escuchadores de eventos de entrada (binding de entrada): 7

6. 6. Validación de formularios

100 / 100

Cumplido. Se realizan validaciones robustas tanto en backend (Django Forms) como en frontend.

- Formularios de Django estructurados (forms.py): 4
- Llamadas a validación segura en vistas (`.is_valid()`): 14
- Atributos de validación HTML5 en plantillas frontend: 25
- - .\pago\forms.py

- - .\pedidos\forms.py

- - .\reservas\forms.py

- - .\usuarios\forms.py

7. 7. Configuración limpia por Variables de Entorno

10 / 100

Datos sensibles expuestos críticamente en el repositorio. Falta el uso de variables de entorno.

- Archivo .env o .env.example presente: No

- Llamadas a variables de entorno en código (os.getenv/decouple): 5

- [ALERTA] Credencial expuesta: SECRET_KEY expuesta en settings.py

8. 8. Jerarquía y organización del código fuente

100 / 100

Estructura de directorios altamente organizada bajo el estándar de aplicaciones Django MVT.

- - vistas (views.py): 5

- - modelos (models.py): 4

- - formularios (forms.py): 4

- - rutas (urls.py): 5

- - plantillas (.html): 39

- - recursos estaticos (.css/.js): 7

- - tests (tests.py): 4

9. 9. Pruebas unitarias sobre componentes

100 / 100

¡Excelente cobertura de pruebas! Se encontraron 65 métodos de prueba estructurados.

- Archivos de prueba detectados: 4

- Clases de Test definidos: 12

- Funciones de prueba (test_*): 65

- - .\pagotests.py

- - .\pedidos\tests.py

- - .\reservas\tests.py

- - .\usuarios\tests.py

10. 10. Optimización de carga (Lazy loading / split)

75 / 100

Optimización parcial. Añade loading='lazy' en imágenes pesadas de tus catálogos o galerías.

- Imágenes o recursos con carga diferida (loading='lazy'): 1
- Uso de scripts asíncronos o diferidos (async/defer): 0

11. 11. Documentación (JSDoc / Comentarios)

80 / 100

Nivel de documentación adecuado, pero se recomienda documentar clases y funciones complejas.

- Archivos de código escaneados: 42 Python, 3 JavaScript
- Docstrings de documentación en Python: 40
- Comentarios estilo JSDoc en JavaScript: 0

12. 12. Adaptabilidad de la interfaz (Responsive)

100 / 100

Diseño completamente responsivo y adaptable mediante Bootstrap Grid y consultas de medios CSS.

- Media Queries detectadas en hojas de estilo CSS: 5
- Uso de contenedores y rejillas responsivas (Bootstrap Grid/Flexbox): 87

RECOMENDACIONES CLAVE PARA MEJORA:

- Migrar las variables SECRET_KEY y EMAIL_HOST_PASSWORD a variables de entorno (.env).
- Reducir el uso de estilos en línea (style="...") en las plantillas HTML (trasladar a CSS).
- Añadir loading="lazy" a imágenes y async/defer a scripts.

- Mejorar documentacion de metodos complejos del frontend usando estandar JSDoc.